Machine Learning Project

Master degree in Artificial Intelligence

Giorgio Chelli
g.chelli4@studenti.unipi.it

Massimo Monai
m.monai@studenti.unipi.it

Type of project: **A with CM**

Date:
23/01/2026

Objective

This project aims to explore and evaluate the use of Deflected Subgradient Descent (SSD), an optimization method designed to address problems where the objective function is non-differentiable.

The subgradient descent technique allows the descent process to be performed even in the presence of sharp points or discontinuities in the derivative. This approach has some critical issues: it is characterized by a large number of hyperparameters, requires careful calibration, and is generally less efficient in terms of convergence speed than gradient-based methods.

In the project, Deflected Subgradient Descent is compared with the results obtained using the classical implementation of Gradient Descent with Momentum (GRAD).

All algorithms and numerical experiments were developed using the MATLAB environment.

Neural Network architecture

The adopted model is a fully-connected feedforward Neural Network (Multi-Layer Perceptron).

Layers and activation functions

The architecture consists of an input layer with 12 units, 2 hidden layers with number of neurons selected by model selection, and an output layer with 4 units. The hidden layers employ **Leaky ReLU** activation functions, while the output layer uses **linear** activation.

Weights initialization

- The hidden layers were initialized using **He-Kaiming** initialization, based on the fan-in criterion.
- The output layer, however, was initialized randomly with small-amplitude weights for stability.

Regularization

L1 regularization was used, consistently with the use of the subgradient method, since the L1 norm introduces a non-differentiable objective function but tractable using this approach.

Classic NN: Gradient Descent

The simpler version, called Classic NN, is a Neural Network with the previously described architecture, trained using **Gradient Descent** with **Heavy-Ball momentum** and **L1 regularization**.

Initially, SGD and full-batch implementations were developed using OOP constructs for clarity, but they resulted in very long training times (hours per training cycle). For hyperparameter grid search, a **mini-batch** version was implemented, exploiting MATLAB's vectorized and matrix-based computations, reducing training time to a few seconds.

Training employed an **Early Stopping** criterion based on the normalized validation MEE, with a tolerance of 1% and a patience window of 200–400 epochs (fixated, not selected through grid search).

Deflected Subgradient NN architecture

Batch

Training was performed in batch mode, as commonly used in subgradient methods.

Stopping criteria

An early stopping strategy was adopted, with patience and tolerance parameters selected via grid search during the validation phase.

Learning algorithms

We have developed two models of learning algorithms based on deflected gradient descent, which we compared with the results of classical gradient descent:

1. Deflected Subgradient Descent - Stepsize Restricted
2. Deflected Subgradient Descent - Deflection Restricted

Advanced NN: Deflected Subgradient Descent

Deflected Subgradient Descent extends subgradient descent by combining the current subgradient with information from previous iterations, introducing a deflection term that reduces oscillations., while in deflection restricted,, making the optimization more stable.

One of the main limitations of the method is the need to estimate $f*$, the optimal value of the objective function, which can be difficult or require approximations, affecting the choice of step size and the speed of convergence.

Stepsize restricted

The update step is limited

$$\alpha^i = \beta^i (f^i - f_*) / \| d^i \|^2 \quad \wedge \quad \beta^i \leq \gamma^i$$

Deflection restricted

The deflection contribution is constrained

$$\frac{\alpha^{i-1} \| d^{i-1} \|^2}{(f^i - f_*) + \alpha^{i-1} \| d^{i-1} \|^2} \leq \gamma^i$$

Deflected Subgradient Descent and SGPTL

In our learning algorithms, the estimation of f^* was obtained using the SGPTL stepsizing strategy for adaptive control of the descent direction.

Although this approach introduces an high number of hyperparameters, experimental results show improved convergence smoothness and good validation performance.

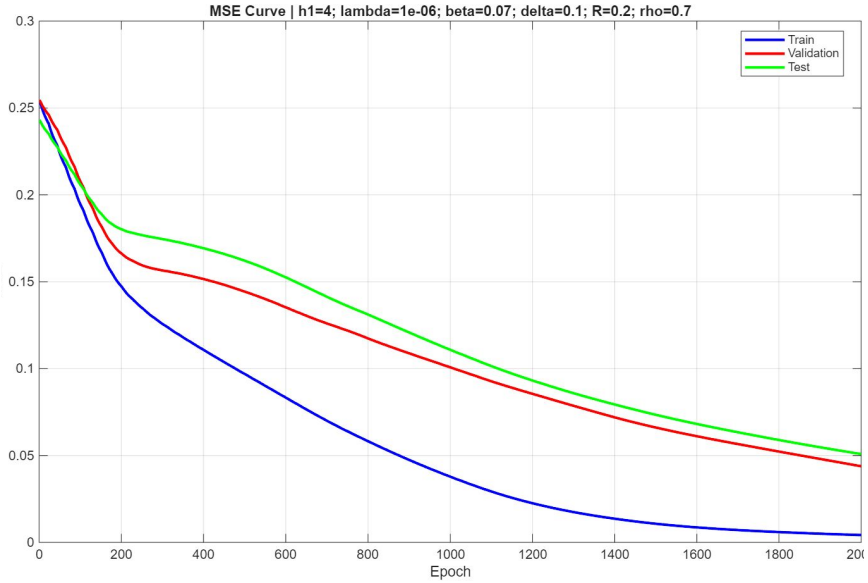
procedure $x = SGPTL\,(f\,,\,x\,,\,i_{max}\,,\,\beta\,,\,\delta_0\,,\,R\,,\,\rho\,)$
  $r \leftarrow 0;\ \delta \leftarrow \delta_0;\ f_{ref} \leftarrow \bar{f} \leftarrow f(x);\ i \leftarrow 1;$
  while($i < i_{max}$) **do**
    $g \in \partial f(x);\ \alpha \leftarrow \beta(f(x) - (f_{ref} - \delta))\,/\,\|g\|^2;\ x \leftarrow x - \alpha g;$
    if($f(x) \leq f_{ref} - \delta\,/\,2$) **then** $\{\, f_{ref} \leftarrow \bar{f};\ r \leftarrow 0;\ \}$
      else if($r > R$) **then** $\{\, \delta \leftarrow \delta\rho;\ r \leftarrow 0;\ \}$
      else $r \leftarrow r + \alpha\|g\|;$
    $\bar{f} \leftarrow \min\{\bar{f}\,,\,f(x)\};\ i \leftarrow i + 1;$

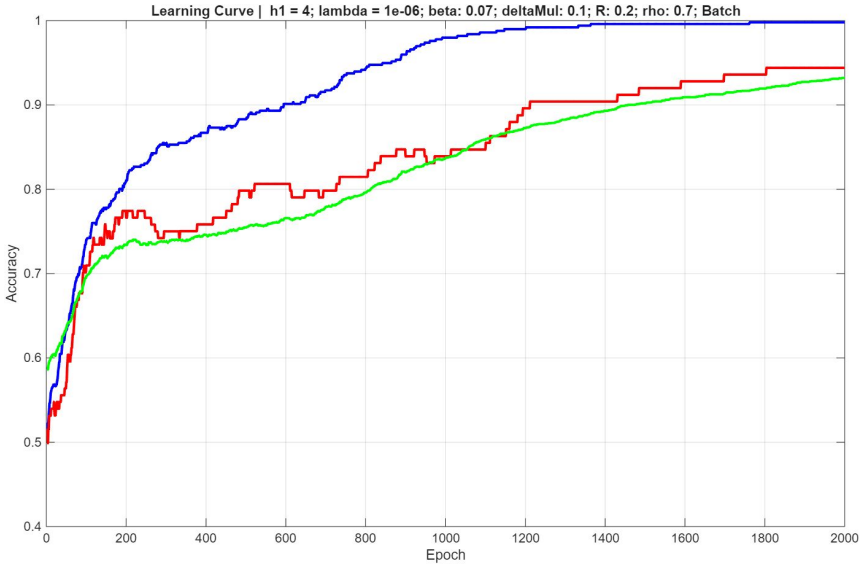
Monk Results

The same learning algorithms was applied to the MONK dataset, but with some modifications to the model structure. The model consists of a single hidden layer, with a **tanh** activation function, and an output layer with a **sigmoid** activation function. The model does not use early stopping but maintains L1 regularization.

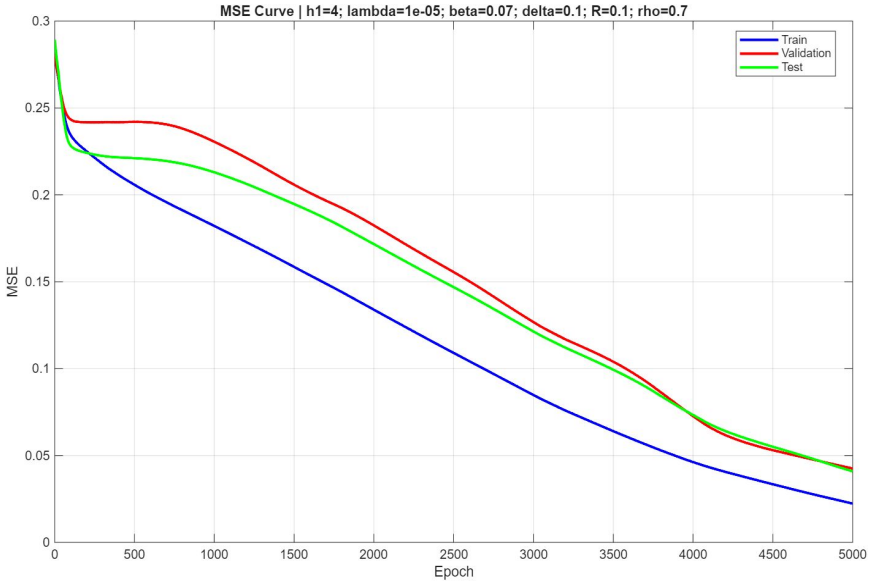
Task	#Units, lambda, beta, delta, R, rho	MSE (TR/TS)	Accuracy(TR/TS)
MONK 1	h1 = 4, λ = 1e-6, β = 0.07, δ = 0.1, R = 0.2, ρ = 0.7	0.0042/0.0508	0.9980/0.9319
MONK 2	h1 = 4, λ = 1e-5, β = 0.07, δ = 0.1, R = 0.1, ρ = 0.7	0.0223/0.0409	0.9719/0.9435
MONK 3	h1 = 4, λ = 1e-4, β = 0.1, δ = 0.1, R = 0.05, ρ = 0.7	0.0485/0.0401	0.9405/0.9671

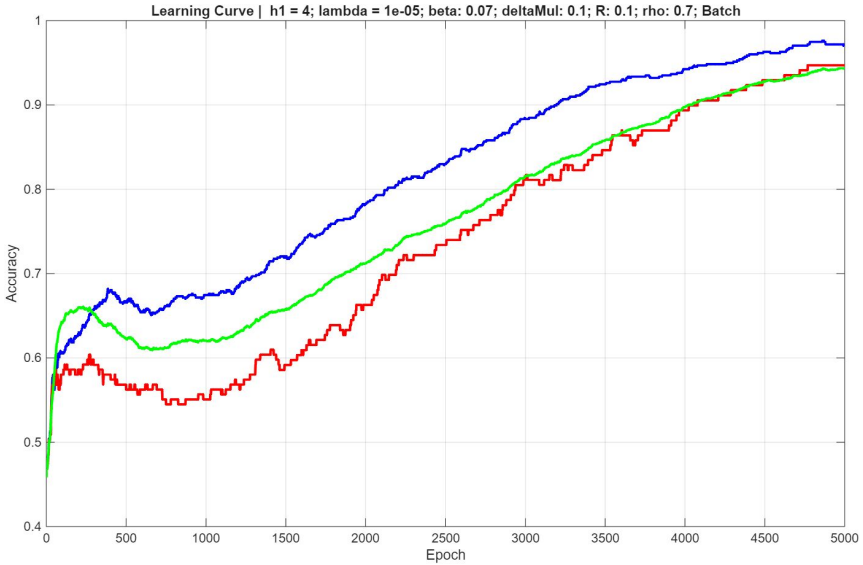
Monk 1


MSE Curve | h1=4; lambda=1e-06; beta=0.07; delta=0.1; R=0.2; rho=0.7
Train
Validation
Test
0.3
0.25
0.2
0.15
0.1
0.05
0
0
200
400
600
800
1000
1200
1400
1600
1800
Epoch

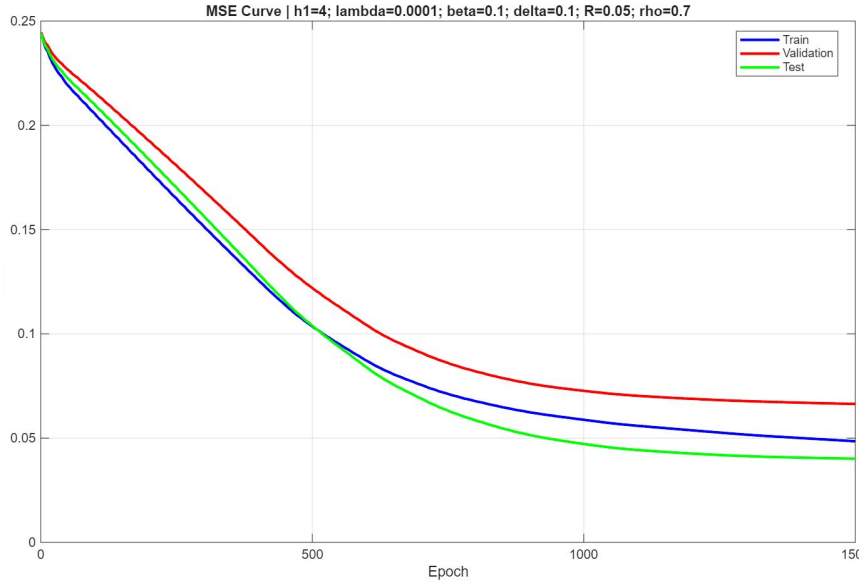

Learning Curve | h1 = 4; lambda = 1e-06; beta: 0.07; deltaMul: 0.1; R: 0.2; rho: 0.7; Batch
Accuracy
1
0.9
0.8
0.7
0.6
0.5
0.4
0
200
400
600
800
1000
1200
1400
1600
1800
2000
Epoch

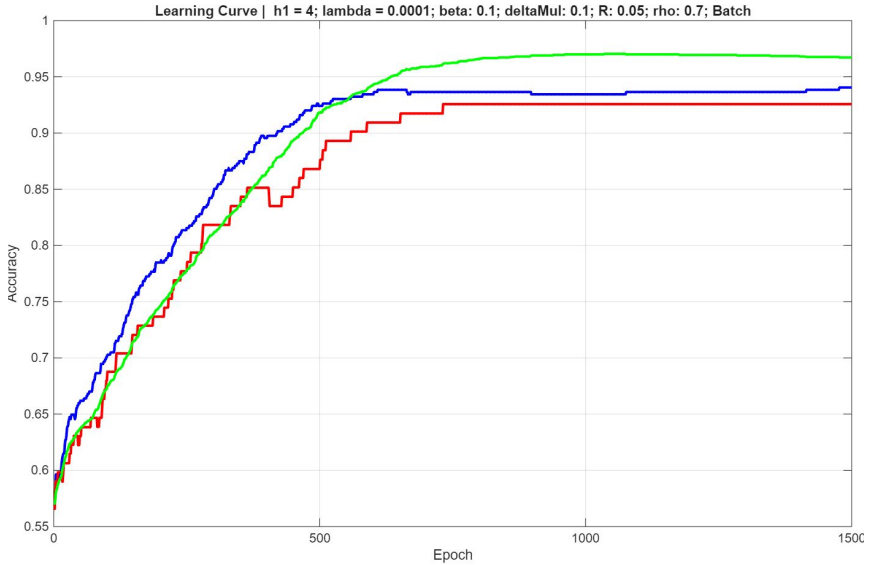
Monk 2


MSE Curve | h1=4; lambda=1e-05; beta=0.07; delta=0.1; R=0.1; rho=0.7
Train
Validation
Test
MSE
Epoch
0
0.05
0.1
0.15
0.2
0.25
0.3
0
500
1000
1500
2000
2500
3000
3500
4000
4500
5000


Learning Curve | h1 = 4; lambda = 1e-05; beta: 0.07; deltaMul: 0.1; R: 0.1; rho: 0.7; Batch
Accuracy
Epoch
0.4
0.5
0.6
0.7
0.8
0.9
1
0
500
1000
1500
2000
2500
3000
3500
4000
4500
5000

Monk 3


MSE Curve | h1=4; lambda=0.0001; beta=0.1; delta=0.1; R=0.05; rho=0.7
Train
Validation
Test
0.25
0.2
0.15
0.1
0.05
0
0
500
1000
Epoch


Learning Curve | h1 = 4; lambda = 0.0001; beta: 0.1; deltaMul: 0.1; R: 0.05; rho: 0.7; Batch
Accuracy
1
0.95
0.9
0.85
0.8
0.75
0.7
0.65
0.6
0.55
0
500
1000
1500
Epoch

CUP Validation schema: data splitting

Training + Validation (64% + 16%) 5-fold-CV	Test (20%) Hold-out

First, we split the data to define the Test set. Then, we perform a 5-fold Cross Validation. In percentages, we have:

- 64% of data for Training
- 16% of data for Validation
- 20% of data for Internal Test

After model selection, retraining was performed on the entire dataset, using the initialized weights and the number of training epochs corresponding to the fold with the best MEE results for the model.

CUP Validation schema: Classic NN

This is the performed grid search:

- **h1 & h2**: number of neurons in the first and second hidden layers, respectively
  - Tested values: h1, h2 ∈ {10-100} at steps of 10
- **eta**: Learning Rate factor
  - Tested values: η ∈ [0.5-0.0001] such as 0.5, 0.1, 0.05, 0.01, 0.005, etc…
- **lambda**: L1 Regularization coefficient
  - Tested values: λ ∈ [1e-2-1e-9] at steps of /10
- **alpha**: Heavy-Ball momentum coefficient
  - Tested values: β ∈ [0.2-0.9] at steps of 0.1
- **batch**: size of batch for the mini-batch algorithm
  - Tested values: mb ∈ [8, 16, 20, 32, 128, 256, 500]

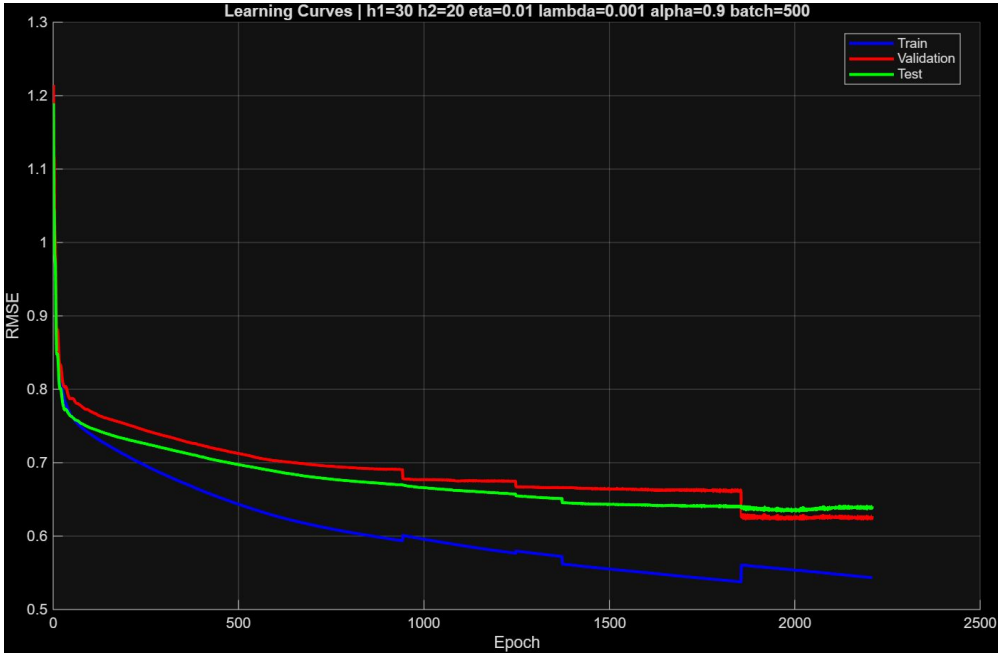
CUP Validation schema: Deflected Subgradient NN

The use of SGPTL and deflected subgradient descent requires, according to the underlying theory, the initialization of a relatively large number of hyperparameters:

- **h1 & h2**: number of neurons in the first and second hidden layers, respectively
  - Tested values: $h1, h2 \in \{10, 20, 30, 40\}$
- **lambda**: L1 Regularization coefficient, promoting sparsity in the network weights
  - Tested values: $\lambda \in [1e\text{-}5, 1e\text{-}3]$
- **beta**: scaling factor, controlling the magnitude of the update along the descent direction
  - Tested values: $\beta \in [0.1, 0.3]$
- **delta**: tolerance parameter used to update the reference function value f_ref
  - Tested values: $\delta \in [1e\text{-}2, 1e\text{-}1]$
- **R**: maximum admissible accumulated displacement without sufficient improvement
  - Tested values: $R \in [1e\text{-}1, 5e\text{-}1]$
- **rho**: contraction factor used to relax the decrease threshold δ when stagnation is detected
  - Tested values: $\rho \in [0.5, 0.8]$

Mini-batch	h1=30, h20=, η=0.01, λ=0.001, α=0.9, mb=500	19.39/22.04/21.58

CUP Results - Classic NN



RMSE norm. (TR/VL/TS):
- 0.54/0.62/0.66

MEE mean (TR/VL/TS):
- 19.39/22.04/21.58

Best MEE per fold:
TR: [20.56,19.93,17.96,19.75,18.73]
VL: [21.48,22.20,22.42,24.20,19.87]
TS: [22.20,21.70,20.66,22.26,21.11]

Training time: 3.2 seconds

Stepsize-restricted	h1=30, h20=, λ=1e-4, β=0.3, δ=0.1, R=0.1, ρ = 0.7	18.91/21.86/22.86

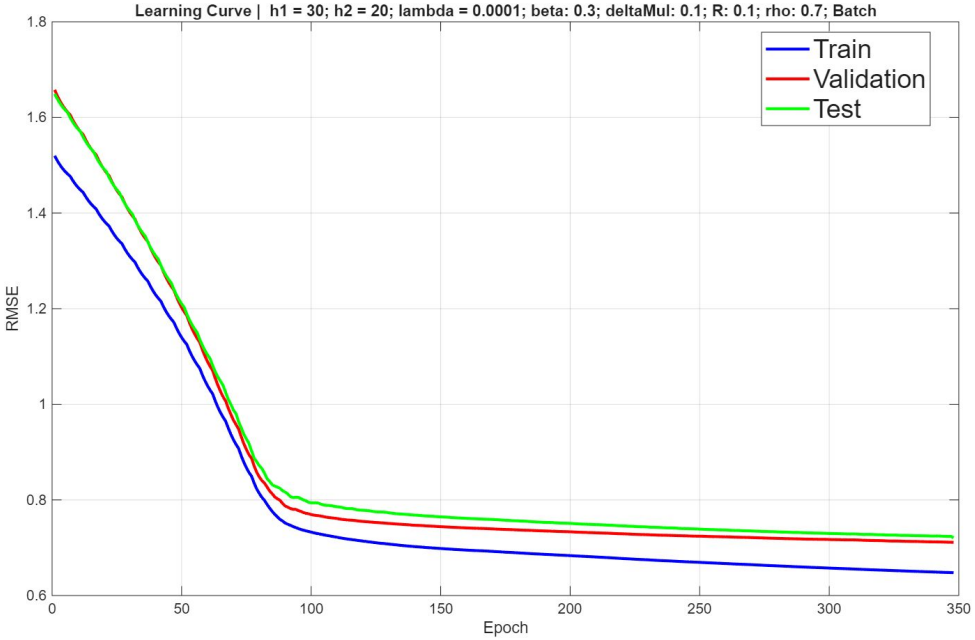
CUP Results - Deflected subgradient NN

The final model was obtained with the stepsize restricted algorithm.

The training time for the final model was approximately 1 hour and 40 minutes.

MEE mean: 18.91/21.86/22.86

RMSE mean: 0.5415/0.6737/0.7087



CUP Results - Deflected NN

The selected model was chosen because it **provides a good trade-off** between accuracy and generalization. Although we are obviously not in a global minima, the gap between TR and VL is **limited**, indicating a controlled fitting on training data and a decent generalization capability.

Although the following is not a criterion for model selection, we can also observe that the test curve is very close and comparable to the validation curve throughout the whole training, confirming its continuous learning on training data and generalization on unseen data. Moreover, each curve generally decreases constantly, converging in a smooth way with stable performances.

Discussion

The deflected subgradient method achieved slightly better performance on the validation set, while showing worse generalization on the test set compared to standard Gradient Descent. This behavior suggests a higher sensitivity to the validation data and potential overfitting effects.

A key practical limitation of the deflected subgradient approach is the large number of hyperparameters involved, whose tuning is nontrivial and strongly affects convergence and generalization. In particular, the method relies on an accurate approximation of the optimal objective value f^*, which is unknown in practice and difficult to estimate reliably, especially in non-convex settings such as neural network training.

In contrast, Gradient Descent, despite its simpler update rule, proved more robust and easier to tune, resulting in more stable test performance.

Conclusions

One very important thing that we grasped from this experience is that **efficiency is key**, since we are talking about very expensive computational processes, and a very efficient algorithm lets you explore more of the feature space.

Another thing that was very surprising to us is the **influence of randomness** in these kinds of processes. A couple of trainings of the same exact model can lead to very different results, mainly because of the random starting point on the loss surface, and also the random splitting of the datasets for training, validation and internal test against which we evaluate the model.

Machine learning requires skills, but also a decent amount of good luck.

BLIND TEST RESULT: Deflection_team_ML-CUP25-TS.csv